

Group Project: Build a Small Product in **8 Weeks**

Team size: 3-6 students

Outcome: Understand what users need, plan the work, build three small working versions (sprints), and explain how your design works in a final presentation.

Terms We Will Use

- **Requirement:** Something the product must do or a quality it must have.
 - **Functional requirement (FR):** A feature. Example: “Users can book an appointment.”
 - **Non-functional requirement (NFR):** A quality (speed, security, usability). Example: “Bookings save in ≤ 2 seconds for 95% of cases.”
- **Persona:** A short, realistic profile of a typical user.
- **Scenario:** A small story of a user completing one task, step by step.
- **User story:** *As a <person>, I want <something> so that <something>.*
- **Acceptance criteria:** Short tests (Given/When/Then) that prove a story is done.
- **Backlog:** Your ordered list of stories.
- **Sprint:** A one-week block of focused work that ends with a demo.
- **Architecture:** A simple map of the main parts of your system and how they talk to each other.

Team Roles

Assign these roles in Week 1 and record them in your **Team Operating Agreement (TOA)**. You may combine roles on small teams but all responsibilities must be covered. **One member can have multiple roles** (e.g. a member can be UX, TL and developer).

1. **Product Owner (PO):** Owns value and clarity. Keeps the backlog in priority order, writes stories with acceptance criteria, and accepts or rejects work at sprint reviews.
2. **Scrum Master (SM):** Owns the process. Runs planning/review/retro, schedules stand-ups, and removes blockers.
3. **Business Analyst (BA):** Owns requirements quality. Plans interviews, organizes notes, writes FRs/NFRs, maintains a simple **traceability** table (Requirement → Story → Test).
4. **UX Designer (UX):** Owns user experience. Creates personas, scenarios, and simple wireframes.
5. **Tech Lead / Architect (TL):** Owns technical coherence. Draws architecture diagrams, writes short Architecture Decision Records (ADRs), reviews code structure.
6. **Developers / QA (everyone codes/tests):** Break stories into tasks, implement vertical slices, write tests from acceptance criteria.

Why roles? Clear ownership reduces confusion. Decisions have an owner. You go faster with fewer arguments.

Tools You Must Set Up

- [Trello](#) with lists: *Icebox*, *Product Backlog*, *Sprint Backlog*, *In Progress*, *In Review*, *Done*.
- **Repo** on GitHub/GitLab with a README (how to run the project).
- **Docs** in Google Docs/Words.

What You Will Do Each Week

You will submit a **weekly package**. Follow the steps in order. If stuck, do the next small step, progress matters more than perfection.

Week 1: Understand the Problem and Organize the Team

You lay the groundwork for Requirements Engineering. You frame the problem and stakeholders (inception), plan how to collect information (elicitation planning), agree on roles and working rules, and set up basic requirements management tools (backlog, traceability). These foundations reduce confusion later and make Week 2 faster.

What to do

1. **Create 2 personas** (UX leads) and **write 3 scenarios** (short task stories) for those personas.
2. **Set up Trello and repo.** Add the lists above. Create one example story card using the template at the end.
3. **Plan 4-6 interviews** for Week 2 (BA leads): pick people, book times, write a question script, include a simple consent line.
4. **Write your Team Operating Agreement (TOA)** (SM facilitates): roles, meeting times, decision rules, conflict resolution, and a small **RACI** table (Responsible/Accountable/Consulted/Informed) for key activities.

Why these matters

You cannot build the right thing until you know who it is for, what they need, and how your team will work together.

What to submit (Week 1 package)

- Product Vision
- 2 Personas + 3 Scenarios
- Trello & repo links
- TOA with a RACI table

Week 2&3: Talk to Users; Turn Findings into Requirements; Plan Sprint 1

You move from guesses to evidence. You perform elicitation (interviews/observation), do analysis (group notes into themes, refine personas and scenarios), create the first specification (FRs/NFRs and user stories with acceptance criteria), and set priorities/estimates (Agile release planning). You also start requirements management by baselining a well-ordered backlog.

What to do

1. **Run the interviews/observations.** Take timestamped notes; do not collect sensitive data.
2. **Sort notes** into themes (affinity mapping). Update personas and scenarios with what you learned.
3. **Write requirements:**
 - **FRs:** list 8-12 features you must support in simple sentences.
 - **NFRs:** list 3-5 qualities with numbers (see examples below).
4. **Build the backlog:** create 12-16 **user stories** with Given/When/Then acceptance criteria. PO orders the backlog using Must/Should/Could.
5. **Estimate** each story using points (1, 2, 3, 5, 8, 13). Use **Planning Poker**. Put the estimate in the Trello card title.
6. **Plan Sprint 1 (one week):** choose 2-3 top stories you believe you can finish. Break each into small tasks. Move them to *Sprint Backlog*.

Why these matters

User input replaces guesses with facts. Clear stories and tests let you know when you are done.

What to submit (Week 2 package)

- Interview notes + a one-page findings summary
- Updated personas & scenarios
- Requirements list (FRs & NFRs)
- Backlog (12-16 stories) with priorities and estimates
- Sprint 1 plan (selected stories + tasks)

Week 4: Build Sprint 1; Review & Retro; Draft Architecture

You run the first full build-verify-validate cycle. You execute Sprint 1 (Agile), do verification using the acceptance-criteria tests, and validate by demoing to stakeholders in review. You also begin architectural design (C4 Context/Container, first ADR) to align implementation with the requirements you discovered.

What to do

Run Sprint 1 (5-6 days)

1. **Daily stand-up (10-15 min)**: Yesterday / Today / Blockers. Move cards across the board.
 - **Build 2-3 stories** end-to-end (even if the data is mocked).
 - **Review (demo)** on the last day. PO accepts/rejects stories using acceptance criteria.
 - **Retrospective (30-45 min)**: What went well? What will we change next sprint?
2. **Draft the architecture** (TL leads): Create **C4 Context and Container diagrams**; write **1 ADR** for a key decision you already made (e.g., SPA vs server-rendered).

Why these matters

A working slice shows real progress. A first architecture map keeps the system understandable as it grows.

What to submit (Week 3 package)

- Sprint 1 review notes (accepted/rejected stories)
- Daily stand-up notes (Yesterday / Today / Blockers)
- C4 Context + Container diagram (images or links)
- 1 ADR

Week 5&6: Build Sprint 2; Add Details to Architecture; Define Quality Checks

You mature the design and start treating non-functional requirements explicitly. You execute Sprint 2, add a Component view and more ADRs, and write quality attribute scenarios with checks (performance, security, usability). This is requirements validation for qualities and continued requirements management as priorities change.

What to do

1. **Plan Sprint 2** using new priorities from the review.
2. **Build Sprint 2** with **2-4 stories**. Keep tests tied to acceptance criteria.
3. **Add a Component diagram (C4-3)** for one critical area (e.g., authentication, search).
4. **Write 1-2 more ADRs** for decisions made this sprint.
5. **Choose 3-5 quality attributes** (performance, security, usability, reliability, accessibility) and write **simple checks** with numbers (see examples below). Start collecting evidence (screenshots, timings).

Why these matters

Architecture and quality do not appear by magic; you plan and check them on purpose.

What to submit (Week 4 package)

- Sprint 2 review notes
- Updated C4 diagrams (add one Component)
- ADRs (now 2–3 total)
- List of quality attributes and your checks (with early evidence)

Week 7: Build Sprint 3; Validate with Users; Connect Requirements to Tests

You prove coverage and value. You run Sprint 3, do user validation sessions, and build a traceability table linking **Requirement → Story → Test → Evidence**, core requirements management. You draft the presentation so that your narrative clearly connects needs, design, and results.

What to do

1. **Plan and run Sprint 3** (2-3 more stories). Focus on stability and polish.
2. **Validate with 2 non-team users**: show the latest build; collect 4-6 feedback bullets and record changes you made.
3. **Create a Traceability Table** that links **Requirement → Story → Test → Evidence** (screenshot, URL, or log).
4. **Presentation prep (first draft)**: build your slides and demo plan using the Presentation Requirements below. Do a timed rehearsal.

Why these matters

Validation proves you built the right thing. Traceability proves you can show evidence for each requirement.

What to submit (Week 5 package)

- Sprint 3 review notes
- Validation summary and list of changes
- Traceability table (first complete draft)
- Slide deck draft (PDF) and demo plan

Week 8: Final Presentation & Submission

You communicate outcomes and close the project. This week emphasizes validation (showing that the product meets needs), management (final baselines for artifacts), and process reflection (lessons learned). Your presentation is your stakeholder-facing specification of what you built and why it matters.

What to do

1. **Polish:** ensure every selected story meets **Definition of Done** (implemented, reviewed, tests pass, demoed, docs updated).
2. **Finalize slides and demo** following the **Presentation Requirements** below.
3. **Deliver your live presentation** in class. Upload final materials.

What to submit (Week 6 package)

- Final slide deck (PDF)
- Final C4 diagrams + ADRs
- Final traceability table
- Individual “Lessons Learned” (one page or half)

Presentation Requirements (Week 8)

Time: 12 minutes talk + 3 minutes Q&A.

Team speaking: All required

Slides (max 12):

1. Title (team, project name)
2. Problem & who it affects (personas)
3. Scenarios (one slide, bullets)
4. Requirements (top FRs + NFRs with numbers)
5. Backlog & plan (what you actually built over 3 sprints)
6. Architecture (C4 Context + Container on one slide; optional tiny Component inset)
- 7–8. Demo plan (story A and story B) with 1-2 acceptance criteria each
9. Quality checks (performance/security/usability evidence)
10. Validation & traceability (table snippet: Requirement → Story → Test → Evidence)
11. Lessons learned (roles, what helped)
12. Roadmap (what you would do next)

Live demo tips: Have sample data ready and a recorded backup (gif/video) in case of Wi-Fi issues.

Grading for the presentation (50 pts of the total):

- Clear problem & users (10)
- Requirements & backlog explained simply (10)
- Architecture & quality checks are accurate and brief (10)
- Demo maps to acceptance criteria and works (10)
- Timing and teamwork (speakers balanced, QCA handled) (10)

Grading Rubric (100 pts)

- **Weekly delivery (5 pts per week, 30 pts total):** graded by percentage of valid submissions. For example, group submitted **4 out of 5** items, weekly grade is:

$$\frac{4}{5} \times 5 = 4$$

- **Presentation (50 pts):** stated above.
- **Working software demos (15 pts):** any demo to the FRs.
- **Tests or CI (5 pts):** shown in demos.

How to Create Your First Trello Story Card

Title: Search events by keyword (Persona: Maya)

Description:

- **User story:** As a *busy student (Maya)*, I want to search events by keyword so that I can quickly find relevant events.
- **Acceptance criteria:**
 - Given I am on the Events page, When I type “career” and press Search, Then I see events with “career” in the title or description.
 - Given there are no matches, When I search “golf”, Then I see the message “No events found.”
- **Links:** Persona *Maya*, Scenario *Find an event*, Wireframe v1.
- **Tasks:** Create search input; API endpoint; filter results; empty-state message; write tests.
- **Definition of Done:** Code reviewed; tests pass; demo recorded; docs updated; moved to **Done**.
- **Label(s):** Must, Search, UX.

Examples of Small Project Ideas

- Campus event finder with reminders
- Nail salon appointment booking
- Cafeteria pre-order & pickup queue
- Peer-to-peer book exchange
- Attendance & participation tracker

Templates

Persona: Name; Goals; Pains; Behaviors/Environment; Tech Comfort; Key Quotes.

Scenario: Title; Trigger; Actor; Preconditions; Main Steps; Alternate Steps; Success Criteria.

User Story: As a <person>, I want <something> so that <something>.

Acceptance Criteria: 2-4 lines of Given/When/Then.

NFR Example: Performance ($\leq 2s$ p95); Security (passwords hashed; input validated); Usability (keyboard reachable; labels linked to inputs).

ADR: Title; Status; Context; Decision; Alternatives; Consequences.

Traceability Table (header):

Req ID	Type	Persona/Scenario	Story ID	Test ID	Evidence Link
--------	------	------------------	----------	---------	---------------